

AEGIS-CLIMATE

Autonomous Multi-Agent Environmental Intelligence Platform

Translating Environmental Threats into Actionable Business & Civic Intelligence

■ HACKATHON

■ 6-HOUR SPRINT

■ PRODUCTION-GRADE

■ 3 AI AGENTS

Built on DevBlueprint • Nishant Maurya

Technical Architecture • Agent Specs • Tech Stack • Frontend Design • Build Timeline

Table of Contents

- 1. Problem Statement & Vision
- 2. System Architecture Overview
- 3. MCP Hub — The Orchestration Layer
- 4. Agent 1 — Acute Physical Risk Agent
- 5. Agent 2 — Chronic Climate Vulnerability Agent
- 6. Agent 3 — Macro Transition & Financial Risk Agent
- 7. Inter-Agent Data Flow Diagram
- 8. Complete Tech Stack
- 9. Database Design
- 10. REST API Endpoints
- 11. Frontend Architecture & UI/UX
- 12. 6-Hour Hackathon Build Timeline
- 13. DevBlueprint Integration Map
- 14. Deployment & DevOps
- 15. Security Considerations

1. Problem Statement & Vision

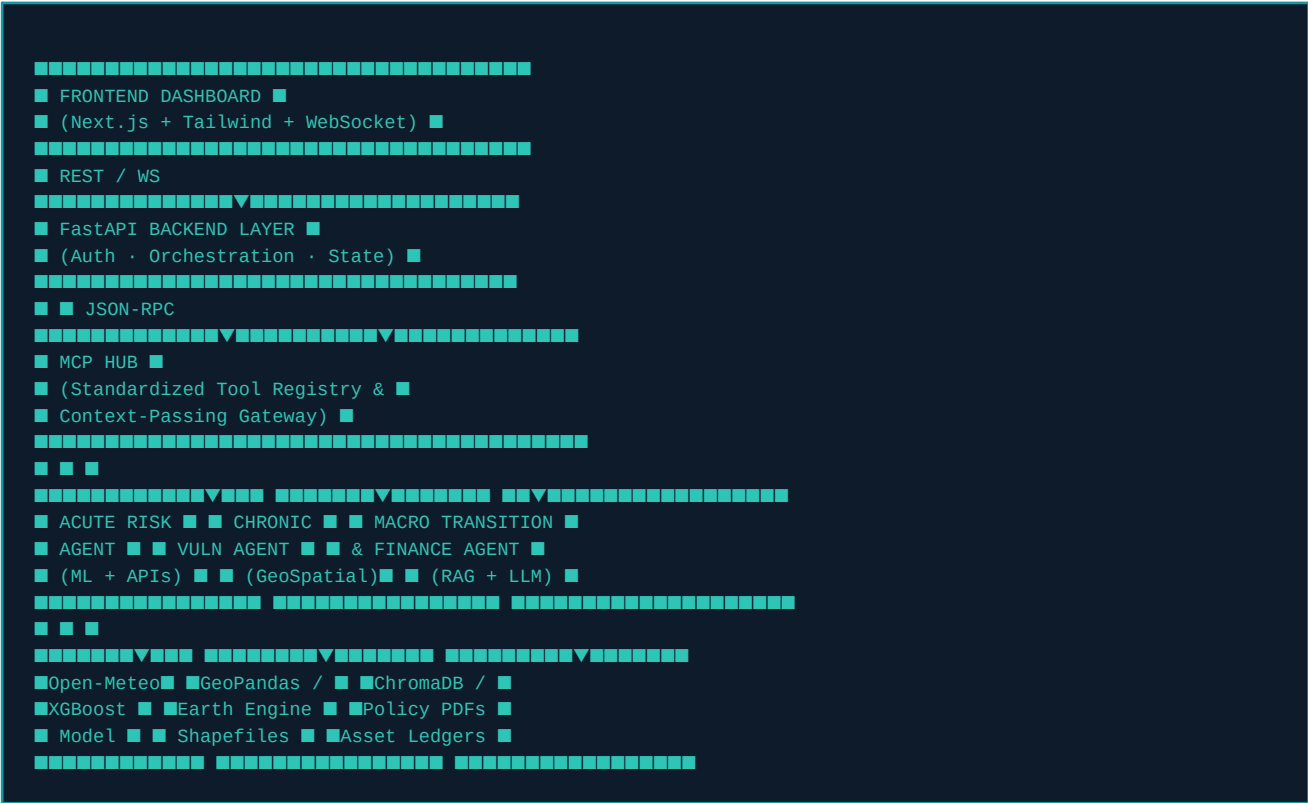
The Gap: Raw environmental data exists everywhere — but no unified autonomous system translates it into real-time, decision-grade business and civic intelligence.

Organizations today face a critical disconnect: petabytes of satellite imagery, meteorological data, and regulatory documents exist — yet decision-makers receive fragmented, delayed, and non-actionable reports. Aegis-Climate bridges this gap by deploying three specialized AI agents that together cover the complete spectrum from immediate weather hazards to 30-year infrastructure exposure to enterprise financial risk under climate regulations.

Problem	Impact	Aegis Solution
No real-time flood probability for local infrastructure	Emergency services respond too late	Acute Agent: ML classifier on live meteo data
No structural vulnerability mapping at scale	Billions in infrastructure loss post-disaster	Chronic Agent: Geospatial layer over hazard zones
Enterprises can't quantify climate regulation risk	Stranded assets, carbon penalty exposure	Transition Agent: RAG on policy + asset data

2. System Architecture Overview

Aegis-Climate uses a Hub-and-Spoke multi-agent architecture connected through the Model Context Protocol (MCP). The MCP Hub acts as the central nervous system — a standardized JSON-RPC gateway that routes tool calls, shares state between agents, and manages asynchronous communication. Each agent is a fully autonomous microservice that can be triggered independently or as part of a cascading pipeline.



Key architectural decisions: Agents communicate exclusively through the MCP Hub — never peer-to-peer. This ensures audit trails, enables replay, and allows individual agents to be upgraded independently. The state graph (LangGraph) manages conditional routing: an alert from Agent 1 auto-triggers Agent 2, whose output feeds Agent 3's financial model.

3. MCP Hub — The Orchestration Layer

The Model Context Protocol Hub is the backbone of Aegis-Climate. It solves the fundamental problem of context fragmentation in multi-agent systems. Instead of each LLM maintaining separate hardcoded API connectors, the MCP Hub acts as a unified protocol gateway.

MCP Component	Role	Tech
Tool Registry	Catalogs all available tools (weather fetch, ML predict, geo-query, RPA search, etc.) with JSON Schema	Pydantic + SQLAlchemy
Context Store	Shared session state — passes a 'risk_event' object between agents	Redis (TTL-based)
Message Bus	Async JSON-RPC 2.0 messages ensure agents don't block each other	asyncio + aiohttp
Router / Orchestrator	LangGraph StateGraph decides conditional triggers: HIGH_RISK → LangGraph 02+	LangGraph 02+
LLM Gateway	Single connection point to Claude/GPT for all agents — centralizes token management	Anthropic SDK / OpenAI

■ MCP Hub runs as a standalone Python microservice on port 8001. All agents call it via HTTP POST to /mcp/invoke with a JSON-RPC payload. The hub validates the tool schema, dispatches, and returns results.

4. Agent 1 — Acute Physical Risk Agent (Operational Layer)

■ Agent 1: Acute Physical Risk Agent		LAYER 1 — OPERATIONAL / REAL-TIME
■ WHAT IT DOES	Calculates immediate, event-driven environmental hazards — specifically localized flash flood probability for any given GPS coordinate. Operates in near-real-time (refresh cycle: 15 min).	
■ HOW IT WORKS	Acts as an autonomous tool-caller. On activation: (1) Fetches live environmental vectors from Open-Meteo API — hourly precipitation, antecedent rainfall (7-day rolling), humidity, soil moisture saturation index. (2) Reformats into a structured feature matrix. (3) Pipes into a locally trained XGBoost/Random Forest classifier via model.predict_proba(). (4) Returns a granular probability score [0.0–1.0] with a risk tier label (LOW / MEDIUM / HIGH / CRITICAL).	
■ CONNECTS TO	Upstream: FastAPI backend + MCP Hub. Downstream: Chronic Agent (if score > 0.65), Frontend Dashboard (WebSocket push), PostgreSQL (event log).	
■ INPUTS	GPS coordinates (lat, lon), timestamp, optional radius_km (default 10km). Live fetched: precipitation, humidity, soil moisture, elevation.	
■ OUTPUTS	JSON: { risk_score: 0.87, tier: 'HIGH', location: {...}, dominant_factor: 'soil_saturation', triggered_chronic: true, timestamp: '...' }	
■ TECH STACK	Python 3.11 · scikit-learn / XGBoost · Open-Meteo API · NumPy / Pandas · FastAPI (tool endpoint) · Pickle model serialization	

Training Data Pipeline

Historical rainfall datasets from NOAA GHCN / ERA5 Reanalysis. Label engineering: flood events sourced from EM-DAT disaster database. Class imbalance handled via SMOTE oversampling. Evaluation targets: Precision ≥ 0.82, Recall ≥ 0.79, F1 ≥ 0.80 on the critical-tier class.

Feature	Source	Importance Rank
soil_moisture_saturation_idx	Open-Meteo hourly	⭐⭐⭐⭐⭐
rainfall_last_24h_mm	Open-Meteo hourly	⭐⭐⭐⭐⭐
rainfall_7d_rolling_mm	Open-Meteo historical	⭐⭐⭐⭐
relative_humidity_pct	Open-Meteo hourly	⭐⭐⭐
elevation_m	DEM static lookup	⭐⭐⭐
drainage_capacity_index	Civic DB / static	⭐⭐

5. Agent 2 — Chronic Climate Vulnerability Agent (Structural Layer)

■ Agent 2: Chronic Climate Vulnerability Agent		LAYER 2 — STRUCTURAL / LONG-HORIZON
■ WHAT IT DOES	Evaluates slow-onset, long-term geographic degradation and infrastructure structural exposure over a 10–30 year horizon. Activates automatically when Agent 1 emits a HIGH or CRITICAL risk score.	
■ HOW IT WORKS	Handles spatial data engineering. Takes the hazard coordinate from Agent 1 and: (1) Triggers GeoPandas tools to load Digital Elevation Models (DEMs) and drainage shapefiles. (2) Overlays hazard zone with civic drainage capacity logs and building material thresholds. (3) Classifies whether the threat causes superficial waterlogging or complete infrastructure failure. (4) Generates a 10/20/30-year exposure matrix with degradation curves.	
■ CONNECTS TO	Upstream: MCP Hub (receives risk_event from Agent 1). Downstream: Agent 3 (passes exposure_report), PostgreSQL (spatial event store), Frontend (geo-layer overlay).	
■ INPUTS	risk_event object from Agent 1 (lat/lon, risk_tier). Geo data: DEM rasters, building shapefiles, drainage capacity CSVs, soil type maps.	
■ OUTPUTS	JSON: { exposure_tier: 'INFRASTRUCTURE_FAILURE', waterlogging_depth_cm: 85, buildings_at_risk: 342, drainage_overflow_pct: 78, 10yr_damage_usd_est: 12400000, geo_overlay_url: '/api/geo/layer/event-xyz' }	
■ TECH STACK	Python · GeoPandas · Shapely · Rasterio · Google Earth Engine API (optional) · PostGIS-enabled PostgreSQL · Folium (geo viz) · NumPy	

Geospatial Analysis Pipeline

Step	Description
Coordinate Ingestion	Receives lat/lon from Agent 1's risk event. Validates against available DEM coverage.
DEM Loading	Loads 30m SRTM Digital Elevation Model tiles for the target radius using Rasterio.
Drainage Overlay	Intersects hazard zone with civic drainage capacity shapefiles (GHSL/OSM data).
Building Exposure	Queries building footprint polygons (OpenStreetMap Overpass) — counts structures inside flood inundation model.
Structural Classification	Rule engine: IF drainage_overflow_pct > 70 AND elevation_delta < 5m → INFRASTRUCTURE_FAILURE tier.
Damage Estimation	Applies HAZUS-MH depth-damage curves per building type to estimate USD exposure.

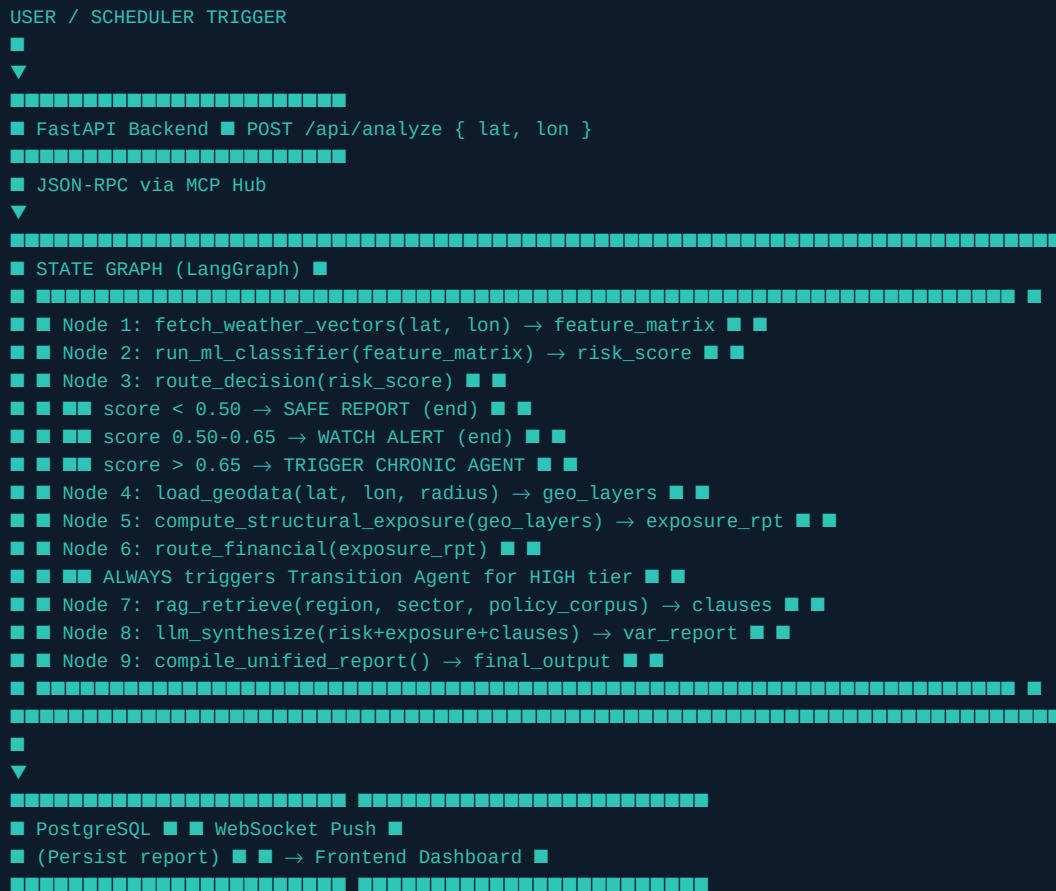
6. Agent 3 — Macro Transition & Financial Risk Agent (Executive Layer)

■ Agent 3: Macro Transition & Financial Risk Agent		LAYER 3 — EXECUTIVE / REGULATORY
■ WHAT IT DOES	Audits regulatory postures and computes enterprise-level financial impacts tied to the low-carbon transition. Ingests policy documents and corporate asset data to model Value at Risk (VaR) from climate regulations like carbon tax schemas and emission bounds.	
■ HOW IT WORKS	Uses a production-grade RAG (Retrieval-Augmented Generation) architecture: (1) Ingests and chunks climate policy PDFs, carbon tax schemas, and emission regulation docs into a vector store. (2) Receives the physical risk + structural exposure report from Agents 1 and 2. (3) Queries vector store to find applicable regulatory clauses for the affected region/sector. (4) Uses LLM (Claude) to synthesize a VaR report — quantifying capital at risk, stranded asset probability, and regulatory compliance gap. (5) Generates an executive-ready briefing.	
■ CONNECTS TO	Upstream: MCP Hub (receives risk_event + exposure_report). Downstream: PostgreSQL (report store), Frontend (executive dashboard), optional: Email/Slack notification.	
■ INPUTS	risk_event (from Agent 1) + exposure_report (from Agent 2) + enterprise asset ledger CSV + regional policy PDF corpus.	
■ OUTPUTS	JSON: { var_estimate_usd: 45000000, stranded_asset_risk: 'HIGH', applicable_regulations: ['Carbon Tax 2026', ...], compliance_gap_pct: 34, recommended_actions: [...], briefing_pdf_url: '/api/reports/exec-xyz.pdf' }	
■ TECH STACK	Python · LangChain / LlamaIndex · ChromaDB (vector store) · PyMuPDF (PDF parsing) · Claude claude-sonnet-4-6 (LLM) · sentence-transformers (embeddings) · FastAPI	

RAG Pipeline Architecture

RAG Stage	What Happens	Tech
Document Ingestion	Policy PDFs parsed → text chunks (512 tokens, 64 overlap) → stored in vector store	PyMuPDF, ChromaDB
Embedding	Chunks converted to dense vectors using sentence-transformers	sentence-transformers, LLM-L6-v2
Retrieval	Cosine similarity search: top-k=5 relevant clauses for the query context	ChromaDB .query()
Augmented Generation	Retrieved clauses + risk data → Claude prompt → structured VaR analysis	Claude-sonnet-4-6
Output Formatting	LLM response parsed into structured JSON with Pydantic validation	Pydantic v2

7. Inter-Agent Data Flow Diagram



■ The state graph is entirely asynchronous — each node is an async Python coroutine. If the Chronic Agent takes more than 30s, the Acute result is streamed to the frontend immediately while the deeper analysis continues in background.

8. Complete Tech Stack

■ AI / ML Layer

Tool / Library	Technology	Purpose
LLM Engine	Claude claude-sonnet-4-6 (Anthropic)	Core reasoning, report synthesis
ML Framework	scikit-learn 1.4 + XGBoost 2.0	Flood classification model
Agentic Framework	LangGraph 0.2 + LangChain 0.3	State graph, tool calling, agent orchestration
RAG / Vector DB	ChromaDB 0.5 + LlamaIndex	Policy document retrieval
Embeddings	sentence-transformers (all-MiniLM-L6-v2)	Text vectorization for RAG

■ Geospatial Layer

Tool / Library	Technology	Purpose
Geospatial Core	GeoPandas 0.14 + Shapely 2.0	Vector geometry operations, shapefile handling
Raster Processing	Rasterio 1.3	DEM loading, raster overlay operations
Earth Engine	Google Earth Engine Python API	Satellite imagery, NDVI, land cover analysis
Geo Visualization	Folium 0.17 + GeoJSON	Interactive map layers for frontend

■ Backend / API Layer

Tool / Library	Technology	Purpose
API Framework	FastAPI 0.112 + Uvicorn	Async REST endpoints, WebSocket support
MCP Protocol	Custom JSON-RPC 2.0 layer + asyncio	Agent communication hub
Database ORM	SQLAlchemy 2.0 + Alembic	DB models, async queries, migrations
Task Queue	Celery + Redis (Broker)	Background task processing for heavy geo ops
PDF Processing	PyMuPDF 1.23	Policy document ingestion for RAG
Validation	Pydantic v2	API request/response schema validation

■ Data / Storage Layer

Tool / Library	Technology	Purpose
Primary DB	PostgreSQL 16 + PostGIS 3.4	Risk events, reports, spatial data storage
Cache / Sessions	Redis 7.2	Context store, agent state TTL, rate limiting
Vector Store	ChromaDB (persistent)	Embedded policy document chunks
Object Storage	MinIO (local) / S3 (prod)	DEM rasters, shapefiles, generated reports

■ Frontend Layer

Tool / Library	Technology	Purpose
Framework	Next.js 14 (App Router)	SSR, API routes, real-time updates
Language	TypeScript 5.x	Full type safety across components
Styling	Tailwind CSS 3 + shadcn/ui	Utility-first design, accessible components
Map Component	Leaflet.js + react-leaflet	Interactive geo-risk map overlay
Charts	Recharts / Chart.js	Risk score timelines, exposure matrices
State	Zustand + TanStack Query v5	Client state + server-state caching
Real-time	WebSocket (native) / Socket.IO	Live agent updates to dashboard

■ DevOps Layer

Tool / Library	Technology	Purpose
Containerization	Docker + Docker Compose	All services containerized
CI/CD	GitHub Actions	Test → Build → Deploy pipeline (from DevBlueprint)
Reverse Proxy	Nginx	SSL termination, load balancing
Monitoring	Sentry + Prometheus + Grafana	Error tracking + metrics

9. Database Design (PostgreSQL + PostGIS)

```
-- Core Tables CREATE TABLE risk_events ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), location
GEOGRAPHY(POINT, 4326) NOT NULL, -- PostGIS point lat DECIMAL(9,6), lon DECIMAL(9,6), risk_score
DECIMAL(4,3), -- 0.000 - 1.000 risk_tier VARCHAR(10), -- LOW/MEDIUM/HIGH/CRITICAL triggered_at
TIMESTAMPTZ DEFAULT now(), agent1_raw JSONB, -- full Agent 1 output status VARCHAR(20) DEFAULT
'PENDING' ); CREATE TABLE structural_reports ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
risk_event_id UUID REFERENCES risk_events(id), exposure_tier VARCHAR(30), buildings_at_risk INTEGER,
damage_estimate_usd BIGINT, geo_layer_url TEXT, agent2_raw JSONB, created_at TIMESTAMPTZ DEFAULT now()
); CREATE TABLE financial_reports ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), structural_id UUID
REFERENCES structural_reports(id), var_estimate_usd BIGINT, stranded_risk VARCHAR(10), regulations
JSONB, -- array of applicable regs briefing_url TEXT, agent3_raw JSONB, created_at TIMESTAMPTZ DEFAULT
now() ); CREATE TABLE policy_documents ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), title
VARCHAR(255), region VARCHAR(100), sector VARCHAR(100), doc_type VARCHAR(50), -- CARBON_TAX /
EMISSION_BOUND / etc source_url TEXT, ingested_at TIMESTAMPTZ DEFAULT now() ); -- Spatial index for
fast geo-queries CREATE INDEX idx_risk_location ON risk_events USING GIST(location); CREATE INDEX
idx_tier ON risk_events(risk_tier);
```

10. REST API Endpoints (FastAPI)

Method	Endpoint	Description
POST	/api/v1/analyze	Trigger full 3-agent pipeline for a coordinate
GET	/api/v1/events/{id}	Fetch a complete risk event with all agent outputs
GET	/api/v1/events?lat=&lon;=&radius;=&tier;=	List events by geo-filter and tier
GET	/api/v1/events/{id}/structural	Fetch Agent 2 structural report for event
GET	/api/v1/events/{id}/financial	Fetch Agent 3 financial VaR report
GET	/api/v1/geo/layer/{event_id}	GeoJSON layer for map overlay
POST	/api/v1/documents/ingest	Upload and ingest a new policy PDF into RAG
GET	/api/v1/documents	List all ingested policy documents
WS	/ws/events	WebSocket — live push for new risk events
GET	/api/v1/health	Health check for all services (DB, Redis, MCP hub)

11. Frontend Architecture & UI/UX (Next.js 14)

The dashboard is a real-time, data-dense interface designed for environmental analysts, civic authorities, and enterprise risk officers. It features three primary views: a live geo-risk map, an event deep-dive panel, and an executive financial briefing module.

Page / Route Structure

/	Landing + live risk map with real-time WebSocket event overlays
/dashboard	Main analyst dashboard — event feed, filters, KPI cards
/events/[id]	Event deep-dive: all three agent outputs in tabbed panels
/map	Full-screen Leaflet map with heat-layer, cluster markers, geo filter
/reports/[id]	Printable executive financial briefing (Agent 3 output)
/admin/documents	Policy document management — upload, ingest status
/api/*	Next.js API routes for BFF pattern — proxies to FastAPI backend

Key UI Components

Component	Description
RiskScoreGauge	Animated radial gauge 0–100 with tier color-coding (GREEN/AMBER/RED/CRIMSON)
GeoRiskMap	Leaflet map with GeoJSON flood zone overlay, building exposure markers, clustering
AgentStatusBadge	Live badge showing each agent's state: IDLE / RUNNING / COMPLETE / ERROR
EventFeedPanel	Real-time scrolling list of risk events — WebSocket powered, infinite scroll
ExposureMatrix	Recharts heatmap: 10/20/30yr exposure across building types and risk vectors
VaRBreakdownChart	Stacked bar chart: financial exposure by regulatory category (carbon tax, asset loss, etc)
DocumentIngestCard	Drag-and-drop PDF upload with chunking progress bar and embedding status
ExecutiveBriefingCard	Summary card with color-coded recommendation tiles for C-suite view

Design System

Token	Value	Usage
--color-primary	#0E9AA7 (Teal)	Buttons, links, active states
--color-safe	#2EC4B6 (Green)	LOW risk tier
--color-watch	#F4A261 (Amber)	MEDIUM risk tier
--color-danger	#E76F51 (Red-Orange)	HIGH risk tier
--color-critical	#D62828 (Crimson)	CRITICAL risk tier
--color-bg-dark	#0D1B2A (Navy)	Map panel, agent cards background

--font-sans	Inter Variable	All body text
--font-mono	JetBrains Mono	Coordinates, code, scores
--radius	8px (base)	Cards, badges, buttons

12. 6-Hour Hackathon Build Timeline

This is your battle plan. Each phase builds on the previous, ensuring a working demo at any checkpoint. Prioritize a functional E2E flow over completeness in any single module.

Hour 0:00 – 1:00		PHASE 1: FOUNDATION SETUP
0:00		Repo init from DevBlueprint structure (mkdir backend frontend, copy .env template)
0:10		docker-compose.yml: spin up PostgreSQL + Redis + ChromaDB in one command
0:20		FastAPI skeleton: main.py with /health, /api/v1/analyze endpoints stubbed
0:35		Database: SQLAlchemy models for risk_events table, run alembic migration
0:50		Open-Meteo API test: confirm live weather fetch for a test coordinate works

Hour 1:00 – 2:00		PHASE 2: AGENT 1 — ML CORE
1:00		Load pre-processed historical weather dataset (ERA5 / NOAA CSV)
1:10		EDA + label engineering: define FLOOD = 1 for events with precip > 80mm
1:20		Train XGBoost classifier, tune with cross-validation, check F1 score
1:40		Pickle model to /models/flood_classifier.pkl
1:50		Wrap in FastAPI tool endpoint: POST /api/v1/tools/predict with feature matrix input

Hour 2:00 – 3:00		PHASE 3: MCP HUB + AGENT 2 (GEO)
2:00		Build MCP Hub: async JSON-RPC dispatcher, tool registry dictionary
2:15		LangGraph state graph: Node 1 (fetch weather) → Node 2 (predict) → Node 3 (route)
2:35		Agent 2 skeleton: GeoPandas loads DEM tiles, calculates basic exposure score
2:50		Connect Agent 1 → Agent 2: HIGH risk triggers geo analysis automatically

Hour 3:00 – 4:00		PHASE 4: AGENT 3 — RAG PIPELINE
3:00		Ingest 3–5 sample climate policy PDFs using PyMuPDF → chunk → embed → ChromaDB
3:20		RAG retrieval: top-5 relevant clauses for a query context
3:35		LangChain chain: retrieved clauses + risk data → Claude Prompt → VaR JSON output
3:50		Connect Agent 3 to state graph: receives exposure_report, returns financial briefing

Hour 4:00 – 5:00		PHASE 5: FRONTEND DASHBOARD
4:00		Next.js setup: Tailwind + shadcn/ui init, base layout with sidebar nav
4:15		Main dashboard page: RiskScoreGauge + AgentStatusBadge + EventFeed (mock data first)

4:30	Leaflet map integration: plot risk event coordinates with tier-colored markers
4:45	Wire to real API: fetch /api/v1/events, render live data, WebSocket for updates

Hour 5:00 – 6:00 PHASE 6: INTEGRATION + DEMO POLISH	
5:00	E2E test: POST /analyze → all 3 agents → result on dashboard
5:20	Executive report page: Agent 3 VaR output formatted as a clean briefing card
5:35	Docker Compose final check: all services healthy with one command
5:45	Demo script: prepare 3 test coordinates (low/high/critical risk scenarios)
5:55	README update with setup steps (from DevBlueprint CHANGELOG/TODO templates)

13. DevBlueprint Integration Map

Your DevBlueprint repo (github.com/Niss54/DevBlueprint) provides battle-tested templates for every layer of this project. Here's exactly which template file maps to which Aegis-Climate component:

DevBlueprint File	Aegis-Climate Use	Customization Needed
Prd.md	Project requirements doc — fill in Aegis-Climate user stories and KPIs	Add agent-specific acceptance criteria, demo KPIs
Architecture.md	System design doc — paste the hub-spoke diagram from Section 10	Add MCP Hub specs, agent connection table
Database.md	DB schema doc — document the 4 core tables from Section 9	Fill in PostGIS spatial table specs
API.md	REST API reference — document all 10 endpoints from Section 10	Section 10 Socket /ws/events spec
UI-UX/ folder	Design guide — document design tokens from Section 11	Add Leaflet map component spec
Testing.md	Testing strategy — Unit tests for ML model, integration for MCP agents	Additional E2E test scenarios
Deployment.md	Docker + CI/CD guide — copy the docker-compose from Section 14	Section 14 Hub container config
Security.md	Security checklist — API key management, rate limiting for MCP	Add MCP API key rotation policy
TODO.md	Use as hackathon task tracker — check off Section 12 timeline tasks	Fill in Phase 1-6 tasks verbatim
.env	Environment config — all API keys and connection strings go here	Add OPENMETEO_KEY, ANTHROPIC_KEY, CHROMA_KEY

14. Deployment & DevOps

Docker Compose (Single Command Local Run)

```
version: '3.9' services: postgres: image: postgis/postgis:16-3.4 environment: POSTGRES_DB: aegis_db POSTGRES_USER: aegis POSTGRES_PASSWORD: ${DB_PASSWORD} ports: ["5432:5432"] volumes: [pgdata:/var/lib/postgresql/data] redis: image: redis:7.2-alpine ports: ["6379:6379"] chromadb: image: chromadb/chroma:0.5.0 ports: ["8002:8000"] volumes: [chromadata:/chroma/chroma] mcp-hub: build: ./mcp_hub ports: ["8001:8001"] depends_on: [redis] environment: REDIS_URL: redis://redis:6379 ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY} backend: build: ./backend ports: ["8000:8000"] depends_on: [postgres, redis, mcp-hub, chromadb] env_file: .env frontend: build: ./frontend ports: ["3000:3000"] environment: NEXT_PUBLIC_API_URL: http://localhost:8000 nginx: image: nginx:alpine ports: ["80:80", "443:443"] volumes: [./docker/nginx.conf:/etc/nginx/nginx.conf] depends_on: [frontend, backend] volumes: {pgdata: {}, chromadata: {}}
```

Quick Start Commands

```
$ git clone https://github.com/Niss54/DevBlueprint && mv DevBlueprint aegis-climate && cd aegis-climate
```

```
$ cp .env .env.local && nano .env.local # fill in ANTHROPIC_API_KEY, DB_PASSWORD
```

```
$ docker compose up -d # starts postgres, redis, chromadb, mcp-hub
```

```
$ cd backend && pip install -r requirements.txt && alembic upgrade head
```

```
$ python scripts/train_model.py # trains and saves flood_classifier.pkl
```

```
$ uvicorn main:app --reload --port 8000 # backend dev server
```

```
$ cd ../frontend && npm install && npm run dev # frontend on :3000
```

15. Security Considerations

■ API Key Management	All API keys (Anthropic, Open-Meteo, Earth Engine) stored in .env — never committed to git. Loaded via python-decouple or os.environ. Rotated monthly in production.
■ Rate Limiting	FastAPI slowapi middleware: 10 req/min per IP on /api/v1/analyze (ML inference is expensive). 100 req/min for read endpoints.
■ Input Validation	All API inputs validated with Pydantic v2 models. Coordinates validated: lat ∈ [-90,90], lon ∈ [-180,180]. File uploads: MIME type check + size limit (20MB max for PDFs).
■ LLM Prompt Injection	Agent 3 RAG pipeline sanitizes all retrieved document chunks before injecting into LLM prompt. System prompt hardcodes output format to prevent jailbreaks in financial reports.
■ Database Security	PostgreSQL: parameterized queries via SQLAlchemy (no raw SQL). Connection pooling with max_overflow=10. DB credentials never logged.
■ Frontend XSS	Next.js App Router: DOMPurify sanitizes any user-facing content. No dangerouslySetInnerHTML usage. CSP headers via Nginx config.
■ CORS Policy	FastAPI CORS middleware: only allow origins from configured FRONTEND_URL env var. In hackathon: localhost:3000. In prod: your deployed domain.

AEGIS-CLIMATE • Built by Nishant Maurya • Hackathon Edition • github.com/Niss54/DevBlueprint • All agent outputs are probabilistic models — verify with domain experts before acting on financial or civic risk assessments.